



Worksheet: Understanding Image Self-Supervised Contrastive Learning (DINO-style)

This worksheet helps you understand image self-supervised contrastive learning, using a simplified DINO-style student-teacher paradigm. It's designed to run in Google Colab (GPU recommended).

Learning objectives:

- Understand why multi-view augmentation is used in self-supervised learning.
- Implement a toy student/teacher (DINO-style) training loop with momentum teacher updates.
- Visualize learned representations and design simple experiments.

Run each cell sequentially in Colab. Cells are commented to explain the core ideas.

```
In [22]: ## 1) Preliminaries – install packages and detect GPU

# The following cell installs required packages in Colab. If torch is already
# Note: in Colab this cell may take a few minutes on first run.

# !pip install -q torch torchvision timm scikit-learn matplotlib requests tqdm

# Python imports and device detection
import os
import torch
import torchvision
import torchvision.transforms as T
import timm
import matplotlib.pyplot as plt
import numpy as np
import requests
from PIL import Image
from tqdm import tqdm

print('torch:', torch.__version__)
print('torchvision:', torchvision.__version__)
print('timm:', getattr(timm, '__version__', 'not installed'))

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')
print('Device:', device)

# make sure a small folder exists for images
os.makedirs('images', exist_ok=True)
```

torch: 2.8.0+cu128
torchvision: 0.23.0+cu128
timm: 1.0.22
Device: cuda:0

In []:

```
In [23]: ## 2) Download 5 example images from the web

# We'll download 5 fixed images (picsum) and save them to ./images for reprodu

urls = [
    'https://picsum.photos/id/237/800/600',
    'https://picsum.photos/id/1025/800/600',
    'https://picsum.photos/id/1069/800/600',
    'https://picsum.photos/id/1003/800/600',
    'https://picsum.photos/id/1084/800/600'
]

import io

filenames = []
for i, url in enumerate(urls):
    r = requests.get(url)
    fname = f'images/img_{i+1}.jpg'
    with open(fname, 'wb') as f:
        f.write(r.content)
    filenames.append(fname)
    print('Saved', fname)

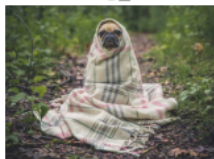
# Display thumbnails
plt.figure(figsize=(12,6))
for i, fname in enumerate(filenames):
    img = Image.open(fname).convert('RGB')
    plt.subplot(1,5,i+1)
    plt.imshow(img)
    plt.axis('off')
    plt.title(f'img_{i+1}')
plt.show()
```

Saved images/img_1.jpg
Saved images/img_2.jpg
Saved images/img_3.jpg
Saved images/img_4.jpg
Saved images/img_3.jpg
Saved images/img_4.jpg
Saved images/img_5.jpg
Saved images/img_5.jpg

img_1



img_2



img_3



img_4



img_5



```

In [36]: # --- 新单元: 定义增强 (color jitter, gaussian blur, solarize, multi-crop)
from PIL import ImageOps
import torchvision.transforms as T
import numpy as np
import torch

# Simple Solarize wrapper using PIL.ImageOps
class Solarize(object):
    def __init__(self, threshold=128):
        self.threshold = threshold
    def __call__(self, img):
        return ImageOps.solarize(img, self.threshold)

# Helper: builds a strong augmentation for global crops
def build_strong_transform(size):
    return T.Compose([
        T.RandomResizedCrop(size=size, scale=(0.2, 1.0)),
        T.RandomHorizontalFlip(p=0.5),
        T.RandomApply([T.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4),
            T.RandomGrayscale(p=0.2)], p=0.5),
        T.RandomApply([T.GaussianBlur(kernel_size=23, sigma=(0.1, 2.0))], p=0.5),
        T.RandomApply([Solarize(threshold=128)], p=0.2),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

# Helper: builds a (weaker) transform for local crops (smaller size)
def build_local_transform(size):
    return T.Compose([
        T.RandomResizedCrop(size=size, scale=(0.05, 0.4)),
        T.RandomHorizontalFlip(p=0.5),
        T.RandomApply([T.ColorJitter(0.4, 0.4, 0.4, 0.1)], p=0.5),
        T.RandomGrayscale(p=0.2),
        T.RandomApply([T.GaussianBlur(kernel_size=7, sigma=(0.1, 1.0))], p=0.3),
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

# Multi-crop generator: returns a tensor of shape (n_views, C, H, W)
def multi_crop_augment(pil_img, n_global=2, n_local=4, global_size=224, local_size=112):
    views = []
    strong_g = build_strong_transform(global_size)
    local_t = build_local_transform(local_size)
    # global crops
    for _ in range(n_global):
        views.append(strong_g(pil_img))
    # local crops
    for _ in range(n_local):
        views.append(local_t(pil_img))
    return torch.stack(views, dim=0) # (n_views, C, H, W)

# Utility: unnormalize tensor -> numpy HWC in [0,1] for plotting
def tensor_to_image_np(tensor):

```

```

# tensor: C,H,W
mean = np.array([0.485,0.456,0.406])[:, None, None]
std = np.array([0.229,0.224,0.225])[:, None, None]
arr = tensor.cpu().numpy()
img = (arr * std + mean).clip(0,1)
img = np.transpose(img, (1,2,0))
return img

# --- 新单元: 对一张图片应用增强并绘图展示 multi-crop 结果
import matplotlib.pyplot as plt

# 确保之前已经下载好了 images 并有 filenames 列表 (在 notebook 前面的单元中创建)
img_path = filenames[0] # 使用第1张图片
img = Image.open(img_path).convert('RGB')

# 生成 views: 2 global + 4 local (示例, 可调)
views = multi_crop_augment(img, n_global=2, n_local=4, global_size=96, local_s
n_views = views.shape[0]

plt.figure(figsize=(12, 3))
for i in range(n_views):
    plt.subplot(1, n_views, i+1)
    v = views[i]
    npimg = tensor_to_image_np(v)
    plt.imshow(npimg)
    plt.axis('off')
    if i < 2:
        plt.title(f'global {i+1}')
    else:
        plt.title(f'local {i-1}')
plt.suptitle(f'Multi-crop augmentations for {img_path}', fontsize=14)
plt.show()

```

Multi-crop augmentations for images/img_1.jpg



In [38]: *## 4) Image augmentations – weak and strong transforms for multi-view*

```

# We'll create two transform factories: a weak transform and a strong transform
import random
from torchvision.transforms import RandomResizedCrop, RandomHorizontalFlip, ColorJitter

# Note: torchvision's GaussianBlur requires kernel size >=1 and odd; we create
from torchvision.transforms import GaussianBlur

# Weak augmentation: small crop + flip
weak_transform = Compose([

```

```

RandomResizedCrop(224, scale=(0.8,1.0)),
RandomHorizontalFlip(p=0.5),
ToTensor(),
Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])

# Strong augmentation: 更强的增强策略
strong_transform = Compose([
    RandomResizedCrop(224, scale=(0.2,1.0)),
    RandomHorizontalFlip(p=0.5),
    ColorJitter(0.4,0.4,0.4,0.1),
    RandomGrayscale(p=0.2),
    GaussianBlur(kernel_size=23, sigma=(0.1,2.0)),
    ToTensor(),
    Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])
])

# Helper to create two views from a PIL image
def get_two_views(pil_img):
    v1 = weak_transform(pil_img)
    v2 = strong_transform(pil_img)
    return v1, v2

# quick visual test of augmentations
img = Image.open(filenamees[0]).convert('RGB')
v1, v2 = get_two_views(img)

plt.figure(figsize=(6,3))
plt.subplot(1,2,1)
plt.imshow(np.transpose((v1.cpu().numpy()*np.array([0.229,0.224,0.225])[:],None
plt.title('view 1 (weak)')
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(np.transpose((v2.cpu().numpy()*np.array([0.229,0.224,0.225])[:],None
plt.title('view 2 (strong)')
plt.axis('off')
plt.show()

```

view 1 (weak)



view 2 (strong)



In [26]: *## 5) Multi-view dataset for contrastive / DINO-style training*

```
class MultiViewDataset(torch.utils.data.Dataset):
    """Returns multiple augmented views per image. Here we return two views (v1, v2)
    Each item: (views_tensor, index) where views_tensor shape = (n_views, C, H, W)"""
    def __init__(self, file_list, n_views=2):
        self.files = file_list
        self.n_views = n_views
    def __len__(self):
        return len(self.files)
    def __getitem__(self, idx):
        path = self.files[idx]
        img = Image.open(path).convert('RGB')
        # 使用 get_two_views(img) 生成两个不同的视图并堆叠
        v1, v2 = get_two_views(img)
        views = torch.stack([v1, v2], dim=0) # shape (2, C, H, W)
        return views, idx

# dataloader
mv_ds = MultiViewDataset(filenamees)
dl = torch.utils.data.DataLoader(mv_ds, batch_size=2, shuffle=True)

# quick iteration to show shapes
for batch_views, batch_idx in dl:
    print('batch_views', batch_views.shape) # (B, n_views, C, H, W)
    break
```

batch_views torch.Size([2, 2, 3, 224, 224])

In [27]: *## 6) Model: simple ResNet backbone + projection head*

```
import torch.nn as nn
from torchvision.models import resnet18

class ProjectionHead(nn.Module):
    def __init__(self, in_dim=512, hidden_dim=512, out_dim=256):
        super().__init__()
        # STUDENT TASK: 实现两层 MLP 投影头, 形式为 Linear(in_dim, hidden_dim) -> Linear(hidden_dim, out_dim)
        # 示例实现已被注释, 学生需要取消注释并理解:
        # self.net = nn.Sequential(
        #     nn.Linear(in_dim, hidden_dim),
        #     nn.GELU(),
        #     nn.Linear(hidden_dim, out_dim)
        # )

        # 为了保持 notebook 可运行, 我们提供一个占位线性层 (功能更弱)
        self.net = nn.Sequential(nn.Linear(in_dim, out_dim))
    def forward(self, x):
        return self.net(x)

class ResNetBackboneWithProj(nn.Module):
    def __init__(self, out_dim=256):
        super().__init__()
```

```

        backbone = resnet18(pretrained=False)
        # remove fc
        backbone.fc = nn.Identity()
        self.backbone = backbone
        self.proj = ProjectionHead(in_dim=512, hidden_dim=512, out_dim=out_dim)
    def forward(self, x):
        # x shape: (B, C, H, W)
        feat = self.backbone(x) # (B, 512)
        z = self.proj(feat)      # (B, out_dim)
        return z

# helper to create student and teacher
def create_student_teacher(out_dim=256):
    student = ResNetBackboneWithProj(out_dim=out_dim).to(device)
    teacher = ResNetBackboneWithProj(out_dim=out_dim).to(device)
    # initialize teacher with student weights
    teacher.load_state_dict(student.state_dict())
    return student, teacher

student, teacher = create_student_teacher()
print('Created student & teacher, param counts:', sum(p.numel() for p in student.parameters()))

```

Created student & teacher, param counts: 11307840 11307840

```

/home/yf25843/anaconda3/envs/qunat/lib/python3.9/site-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
/home/yf25843/anaconda3/envs/qunat/lib/python3.9/site-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=None`.
  warnings.warn(msg)

```

```

In [28]: ## 7.5) Extract initial student representations (before training)
# Collect and store representations from the freshly-initialized student model
# This will be used for "before" visualization so we can compare pre/post training
student.eval()
initial_feats = []
initial_paths = []
with torch.no_grad():
    for fname in filenames:
        img = Image.open(fname).convert('RGB')
        t = basic_transform(img).unsqueeze(0).to(device) # (1,C,H,W)
        z = student(t) # (1, D)
        initial_feats.append(z.cpu().numpy().squeeze())
        initial_paths.append(fname)
initial_feats = np.stack(initial_feats, axis=0)
print('Initial features collected, shape =', initial_feats.shape)
# keep initial_feats in memory for later visualization

```

Initial features collected, shape = (5, 256)

```

In [29]: ## 7) DINO-style loss (simplified) and teacher EMA update

```



```

import torch.nn.functional as F

class DINOLossSimplified:
    def __init__(self, student_temp=0.1, teacher_temp=0.07, center_momentum=0.
        self.student_temp = student_temp
        self.teacher_temp = teacher_temp
        # running center for teacher outputs (for stability)
        self.center = torch.zeros(1, 256).to(device)
        self.center_momentum = center_momentum
    def __call__(self, student_outputs, teacher_outputs):
        """
        student_outputs: (B*n_views, D) logits from student (no softmax yet)
        teacher_outputs: (B*n_views, D) teacher outputs (pre-softmax)
        We treat teacher softmax (with teacher_temp) as target distribution (s
        Then compute cross-entropy: - sum( teacher_probs * log_softmax(student
        """
        # 1) compute teacher probabilities with temperature and centering
        t_out = teacher_outputs.detach() # teacher not backpropagated
        t_out_centered = (t_out - self.center) / self.teacher_temp
        teacher_probs = F.softmax(t_out_centered, dim=-1) # soft targets

        # 2) student log-probabilities with student temperature
        s_logits = student_outputs / self.student_temp
        student_log_probs = F.log_softmax(s_logits, dim=-1)

        # 3) cross-entropy between teacher_probs and student_log_probs
        loss = - (teacher_probs * student_log_probs).sum(dim=-1).mean()

        # 4) update center (moving average) using teacher outputs
        batch_center = t_out.mean(dim=0, keepdim=True)
        self.center = self.center * self.center_momentum + batch_center * (1 -

        return loss

# EMA update for teacher
@torch.no_grad()
def update_teacher(student, teacher, momentum=0.996):
    # For each parameter: teacher = m*teacher + (1-m)*student
    for p_s, p_t in zip(student.parameters(), teacher.parameters()):
        p_t.data.mul_(momentum).add_(p_s.data * (1.0 - momentum))

```

```

In [30]: ## 8) Short toy training loop (run a small number of steps on the 5 images)

# Move models to device and set up optimizer
student = student.to(device)
teacher = teacher.to(device)
optimizer = torch.optim.Adam(student.parameters(), lr=1e-4)
criterion = DINOLossSimplified(student_temp=0.1, teacher_temp=0.07, center_mom

# training hyperparameters
total_steps = int(n_steps) if 'n_steps' in globals() else 60
momentum = 0.996

```



```

from torch.nn.utils import parameters_to_vector

losses = []
param_diffs = []

# Ensure we start training
student.train()
step = 0
# iterate until we reach total_steps
while step < total_steps:
    for batch_views, batch_idx in dl:
        # batch_views: (B, n_views, C, H, W)
        B, n_views, C, H, W = batch_views.shape
        views = batch_views.view(-1, C, H, W).to(device) # (B*n_views, C, H, W)

        # forward student
        student_outputs = student(views)
        # teacher forward (no grad)
        with torch.no_grad():
            teacher_outputs = teacher(views)

        loss = criterion(student_outputs, teacher_outputs)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # EMA update for teacher
        update_teacher(student, teacher, momentum)

        # logging
        losses.append(loss.item())
        vec_s = parameters_to_vector([p for p in student.parameters()])
        vec_t = parameters_to_vector([p for p in teacher.parameters()])
        pdiff = torch.norm(vec_s - vec_t).item()
        param_diffs.append(pdiff)

        if step % max(1, total_steps//5) == 0:
            print(f"step {step}/{total_steps}, loss={loss.item():.4f}, param_diff={pdiff:.4f}")

        step += 1
    if step >= total_steps:
        break

print('Finished toy training')

# After training, extract final representations for visualization
student.eval()
final_feats = []
final_paths = []
with torch.no_grad():
    for fname in filenames:
        img = Image.open(fname).convert('RGB')

```

```

        t = basic_transform(img).unsqueeze(0).to(device)
        z = student(t)
        final_feats.append(z.cpu().numpy().squeeze())
        final_paths.append(fname)
    final_feats = np.stack(final_feats, axis=0)
    print('Final features collected, shape =', final_feats.shape)

    # expose losses/param_diffs for plotting later
    training_stats = {'losses': losses, 'param_diffs': param_diffs}

```

```

step 0/60, loss=1.6393, param_diff=0.3348
step 12/60, loss=6.2987, param_diff=1.1537
step 12/60, loss=6.2987, param_diff=1.1537
step 24/60, loss=6.6860, param_diff=1.5594
step 24/60, loss=6.6860, param_diff=1.5594
step 36/60, loss=5.0369, param_diff=1.7667
step 36/60, loss=5.0369, param_diff=1.7667
step 48/60, loss=4.4442, param_diff=1.8528
step 48/60, loss=4.4442, param_diff=1.8528
Finished toy training
Final features collected, shape = (5, 256)
Finished toy training
Final features collected, shape = (5, 256)

```

In [31]: *## 9) Visualize learned representations (PCA / t-SNE) – before vs after training*

```

# Ensure we have initial_feats and final_feats
if 'initial_feats' not in globals():
    raise RuntimeError('initial_feats not found – run the pre-training extract')
if 'final_feats' not in globals():
    raise RuntimeError('final_feats not found – run the training cell (8)')

from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.metrics.pairwise import cosine_similarity

# PCA side-by-side
pca = PCA(n_components=2)
init_p2 = pca.fit_transform(initial_feats)
final_p2 = pca.transform(final_feats) # use same PCA fit for fair comparison

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.scatter(init_p2[:,0], init_p2[:,1], s=160, c=range(len(initial_paths)), cm
for i,p in enumerate(initial_paths):
    plt.text(init_p2[i,0]+0.01, init_p2[i,1]+0.01, f'img{i+1}', fontsize=10)
plt.title('PCA (before training)')
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.grid(True)

plt.subplot(1,2,2)
plt.scatter(final_p2[:,0], final_p2[:,1], s=160, c=range(len(final_paths)), cm
for i,p in enumerate(final_paths):
    plt.text(final_p2[i,0]+0.01, final_p2[i,1]+0.01, f'img{i+1}', fontsize=10)
plt.title('PCA (after training)')

```

```

plt.xlabel('PC1'); plt.ylabel('PC2'); plt.grid(True)
plt.show()

# t-SNE side-by-side (fit separately but keep same random seed)
tsne = TSNE(n_components=2, random_state=42, perplexity=max(2, min(30, len(initial_feats))))
init_t2 = tsne.fit_transform(initial_feats)
final_t2 = TSNE(n_components=2, random_state=42, perplexity=max(2, min(30, len(final_feats))))

plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.scatter(init_t2[:,0], init_t2[:,1], s=160, c=range(len(initial_paths)), cmap=cm.get_cmap('magma', len(initial_paths)))
for i,p in enumerate(initial_paths):
    plt.text(init_t2[i,0]+0.01, init_t2[i,1]+0.01, f'img{i+1}', fontsize=10)
plt.title('t-SNE (before training)')
plt.grid(True)

plt.subplot(1,2,2)
plt.scatter(final_t2[:,0], final_t2[:,1], s=160, c=range(len(final_paths)), cmap=cm.get_cmap('magma', len(final_paths)))
for i,p in enumerate(final_paths):
    plt.text(final_t2[i,0]+0.01, final_t2[i,1]+0.01, f'img{i+1}', fontsize=10)
plt.title('t-SNE (after training)')
plt.grid(True)
plt.show()

# Cosine similarity matrices and difference (after - before)
sim_init = cosine_similarity(initial_feats)
sim_final = cosine_similarity(final_feats)
sim_diff = sim_final - sim_init

plt.figure(figsize=(15,4))
plt.subplot(1,3,1)
plt.imshow(sim_init, vmin=-1, vmax=1, cmap='viridis')
plt.colorbar()
plt.title('Cosine similarity (before)')
plt.xticks(range(len(initial_paths)), [f'img{i+1}' for i in range(len(initial_paths))])
plt.yticks(range(len(initial_paths)), [f'img{i+1}' for i in range(len(initial_paths))])

plt.subplot(1,3,2)
plt.imshow(sim_final, vmin=-1, vmax=1, cmap='viridis')
plt.colorbar()
plt.title('Cosine similarity (after)')
plt.xticks(range(len(final_paths)), [f'img{i+1}' for i in range(len(final_paths))])
plt.yticks(range(len(final_paths)), [f'img{i+1}' for i in range(len(final_paths))])

plt.subplot(1,3,3)
plt.imshow(sim_diff, cmap='bwr', vmin=-1, vmax=1)
plt.colorbar()
plt.title('Similarity change (after - before)')
plt.xticks(range(len(final_paths)), [f'img{i+1}' for i in range(len(final_paths))])
plt.yticks(range(len(final_paths)), [f'img{i+1}' for i in range(len(final_paths))])
plt.show()

# Training curves (if available)

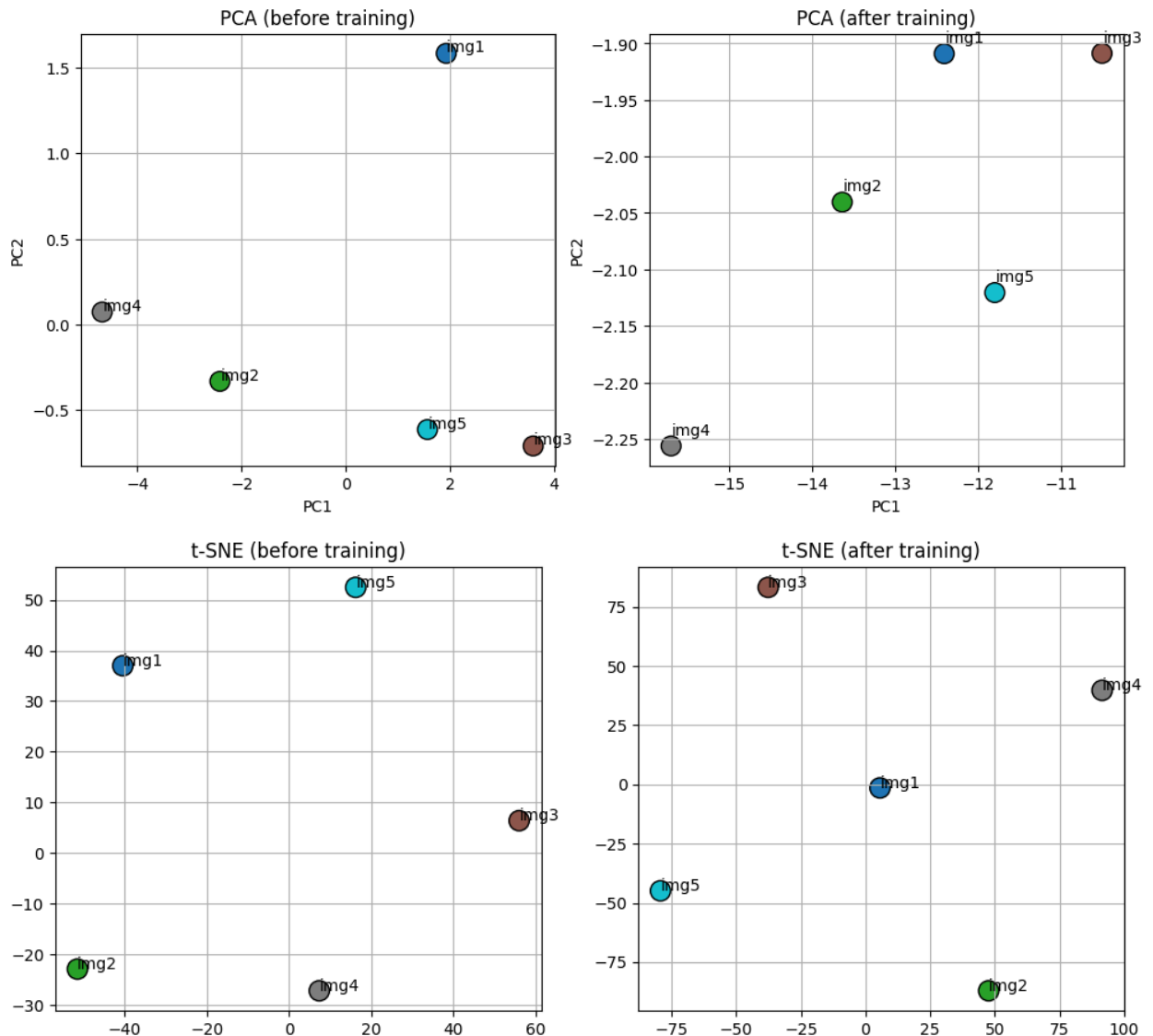
```

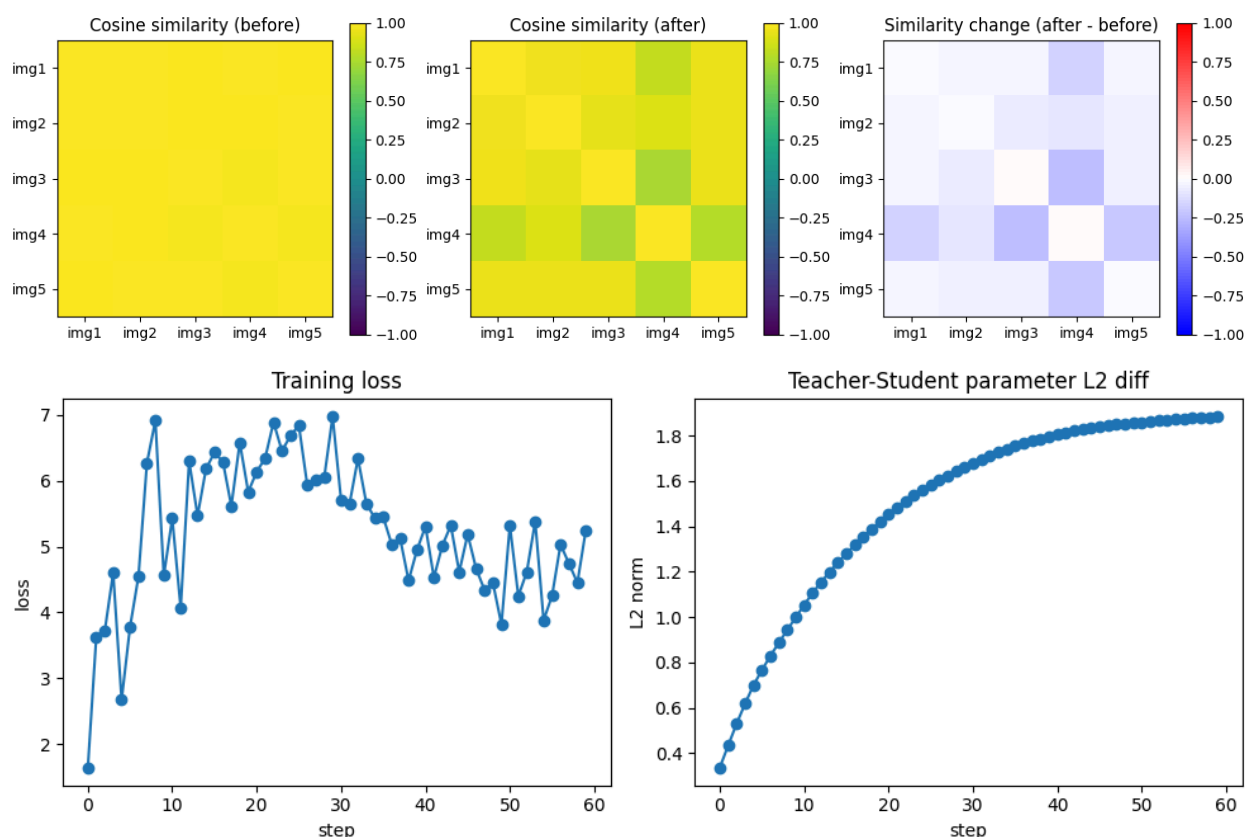
```

if 'training_stats' in globals():
    losses = training_stats.get('losses', [])
    pds = training_stats.get('param_diffs', [])
    plt.figure(figsize=(10,4))
    plt.subplot(1,2,1)
    plt.plot(losses, marker='o')
    plt.title('Training loss')
    plt.xlabel('step')
    plt.ylabel('loss')

    plt.subplot(1,2,2)
    plt.plot(pds, marker='o')
    plt.title('Teacher-Student parameter L2 diff')
    plt.xlabel('step')
    plt.ylabel('L2 norm')
    plt.tight_layout()
    plt.show()
else:
    print('training_stats not found – no training curves to plot')

```





11) How to run this in Colab (short guide)

1. Upload this notebook to Google Colab or open via `File -> Upload notebook`.
2. Run the first cells to install packages and download images. If you are on Colab, enable GPU via `Runtime -> Change runtime type -> GPU` for faster runs.
3. Execute cells sequentially. The toy training loop uses very few steps for pedagogy — to see stronger effects, you can increase `n_steps` and use more images.
4. For student tasks: modify the augmentations / momentum / run the linear probe cells as directed above.

Notes:

- The code implements a simplified DINO-style approach for teaching. The real DINO has additional elements (multi-crop with many local views, multiple prototype heads, more careful temperature scheduling and large-scale training).
- This worksheet is intentionally small-scale so it runs fast in Colab for demonstration and learning.

Enjoy experimenting and feel free to extend the notebook (e.g., add t-SNE visualizations, more images, or a linear probe cell).

In []: